EKF Localization with Landmark Observations Node

IRPP Report

Srivarshitha M 2022102030

Mounika Mylavarapu 2024102006

Haritha Mallige 2024102080

March 7, 2026

Abstract

This report describes the design, implementation, and evaluation of an Extended Kalman Filter (EKF) for two-dimensional mobile-robot localization. A robot that relies exclusively on wheel-encoder odometry accumulates position error over time because small per-step noise integrates without bound. To address this, we implemented a sensor-fusion algorithm that corrects the odometry-based state estimate using sparse range-and-bearing observations of known landmarks. The filter was written from scratch in Python (NumPy) and deployed as a ROS 2 Humble node running inside Docker. Three noise scenarios were evaluated. In the high-noise case the EKF reduced the root-mean-square position error from 0.83 m to 0.50 m ($\approx 40\%$ improvement); in the moderate-noise case the reduction was from 0.32 m to 0.10 m ($\approx 70\%$). All source code is available at https://github.com/16srivarshitha/ekf_localisation_project.

Contents

1 Introduction **3**

2 Mathematical Framework **3**
- 2.1 State and Control Representation . . . 3
- 2.2 Motion Model and Its Jacobians . . . 4
- 2.3 Measurement Model and Its Jacobian . . . 4
- 2.4 The Complete EKF Predict–Update Cycle . . . 5

3 Methodology and Implementation **5**
- 3.1 System Architecture . . . 5
- 3.2 Velocity Derivation from Pose Messages . . . 6
- 3.3 Simulation Parameters . . . 6
- 3.4 Covariance Ellipse Visualisation . . . 7

4 Results and Analysis **8**
- 4.1 Trajectory Overview . . . 8
- 4.2 Odometry Error Growth . . . 10
- 4.3 EKF Update Step Trace . . . 10
- 4.4 Trajectory Accuracy at Key Timesteps . . . 11
- 4.5 RMSE Comparison Across Noise Scenarios . . . 12
- 4.6 Covariance Dynamics . . . 13

5 Conclusion **13**

A Repository Structure **13**

1 Introduction

Robot localization — estimating a robot's pose $[x,\, y,\, \theta]^\top$ in a global reference frame — is a prerequisite for almost every autonomous navigation task. Wheel encoders are a natural first choice for tracking pose because they are cheap, fast, and require no external infrastructure. Unfortunately, every small error in a velocity reading is integrated into the position estimate; over a full lap of a circular trajectory the cumulative drift can exceed a metre even under mild noise conditions, as the baseline comparison in this work confirms.

The Kalman Filter and its nonlinear extension, the EKF, are the classical solution to this problem. Rather than discarding the odometry reading, the EKF treats it as an uncertain prediction and then *corrects* that prediction using sensor observations whose noise characteristics are known. The correction is weighted by the Kalman Gain, which automatically balances how much to trust the prediction versus the sensor depending on the current uncertainty.

This project set out to answer four concrete questions:

1. Can the Jacobians of a nonlinear unicycle motion model and a range-bearing sensor model be derived analytically and implemented efficiently in NumPy?
2. Does the resulting EKF node, integrated into a ROS 2 publish–subscribe architecture, produce a trajectory that visibly and measurably outperforms raw odometry?
3. How does filter performance degrade as process and measurement noise are increased?
4. How does the hand-written filter compare with results obtained from the established `robot_localization` ROS 2 package?

The remainder of the report is structured as follows. Section 2 derives the complete mathematical framework. Section 3 describes the software architecture and implementation decisions. Section 4 presents quantitative results across three noise scenarios. Section 5 draws conclusions and outlines future work.

2 Mathematical Framework

2.1 State and Control Representation

The robot's pose at discrete time t is the three-dimensional vector

$$\mathbf{x}_t = \begin{bmatrix} x_t \\ y_t \\ \theta_t \end{bmatrix}, \tag{1}$$

where (x_t, y_t) is the position in the global frame and θ_t is the heading angle. The control input derived from the wheel encoders is

$$\mathbf{u}_t = \begin{bmatrix} v_t \\ \omega_t \end{bmatrix}, \tag{2}$$

with v_t the linear speed and ω_t the angular speed. In practice, the simulation node publishes raw poses rather than velocities; we recover v_t and ω_t by finite differencing consecutive odometry messages (see Section 3.2).

2.2 Motion Model and Its Jacobians

We use the standard Euler-integration unicycle kinematic model:

$$x_t = x_{t-1} + v_t\,\Delta t\,\cos\theta_{t-1}, \tag{3}$$
$$y_t = y_{t-1} + v_t\,\Delta t\,\sin\theta_{t-1}, \tag{4}$$
$$\theta_t = \theta_{t-1} + \omega_t\,\Delta t. \tag{5}$$

We write this compactly as $\mathbf{x}_t = g(\mathbf{x}_{t-1}, \mathbf{u}_t)$. Because g is nonlinear in θ_{t-1}, the EKF replaces it with a first-order Taylor expansion around the current mean.

State Jacobian G_t

Differentiating g with respect to $\mathbf{x}_{t-1}$:

$$G_t = \frac{\partial g}{\partial \mathbf{x}_{t-1}} = \begin{bmatrix} 1 & 0 & -v_t\,\Delta t\,\sin\theta_{t-1} \\ 0 & 1 & v_t\,\Delta t\,\cos\theta_{t-1} \\ 0 & 0 & 1 \end{bmatrix}. \tag{6}$$

The only off-diagonal coupling is in the third column: a heading error propagates into both x and y predictions at a rate proportional to $v_t\,\Delta t$, which is physically intuitive.

Control Jacobian V_t

Differentiating g with respect to $\mathbf{u}_t$ gives the 3×2 matrix

$$V_t = \frac{\partial g}{\partial \mathbf{u}_t} = \begin{bmatrix} \Delta t\,\cos\theta_{t-1} & 0 \\ \Delta t\,\sin\theta_{t-1} & 0 \\ 0 & \Delta t \end{bmatrix}. \tag{7}$$

V_t maps the control-noise covariance M_t (in velocity space) into state space: $V_t M_t V_t^\top$.

Covariance Prediction

$$\bar{\boldsymbol{\Sigma}}_t = G_t\,\boldsymbol{\Sigma}_{t-1}\,G_t^\top + V_t\,M_t\,V_t^\top. \tag{8}$$

2.3 Measurement Model and Its Jacobian

The environment contains N_L landmarks at known positions $(m_x^{(j)},\, m_y^{(j)})$, $j = 1, \ldots, N_L$. When a landmark is within range, the sensor returns a noisy *range* r and *bearing* ϕ:

$$r = \sqrt{q}, \tag{9}$$
$$\phi = \operatorname{atan2}(m_y - y_t,\; m_x - x_t) - \theta_t, \tag{10}$$

where $q \triangleq (m_x - x_t)^2 + (m_y - y_t)^2$.

Measurement Jacobian H_t

Differentiating $h(\mathbf{x}_t) = [r,\, \phi]^\top$ with respect to $\mathbf{x}_t$:

$$H_t = \frac{\partial h}{\partial \mathbf{x}_t} = \begin{bmatrix} \dfrac{-(m_x - x_t)}{\sqrt{q}} & \dfrac{-(m_y - y_t)}{\sqrt{q}} & 0 \\ \dfrac{m_y - y_t}{q} & \dfrac{-(m_x - x_t)}{q} & -1 \end{bmatrix}. \tag{11}$$

2.4 The Complete EKF Predict–Update Cycle

Step 1. Predict state. $\bar{\mathbf{x}}_t = g(\mathbf{x}_{t-1}, \mathbf{u}_t)$

Step 2. Predict covariance. $\bar{\boldsymbol{\Sigma}}_t = G_t\, \boldsymbol{\Sigma}_{t-1}\, G_t^\top + V_t\, M_t\, V_t^\top$

Step 3. Compute Kalman Gain. $K_t = \bar{\boldsymbol{\Sigma}}_t\, H_t^\top \left(H_t\, \bar{\boldsymbol{\Sigma}}_t\, H_t^\top + R_t\right)^{-1}$

Step 4. Update state. $\mathbf{x}_t = \bar{\mathbf{x}}_t + K_t(\mathbf{z}_t - h(\bar{\mathbf{x}}_t))$

Step 5. Update covariance. $\boldsymbol{\Sigma}_t = (I - K_t H_t)\, \bar{\boldsymbol{\Sigma}}_t$

The term $\mathbf{z}_t - h(\bar{\mathbf{x}}_t)$ is the *innovation* — how much the actual sensor reading differs from what the filter predicted it would be. When the innovation is large the filter trusts the sensor; when it is small the prediction was already accurate. The Kalman Gain K_t automatically scales the correction by the ratio of prediction uncertainty to total (prediction + sensor) uncertainty.

One subtle but important implementation detail: because ϕ is a periodic quantity, the bearing component of the innovation must be wrapped to $(-\pi, \pi]$ after subtraction. Failure to do this causes filter divergence whenever the robot crosses the $\pm\pi$ heading boundary.

3 Methodology and Implementation

3.1 System Architecture

The project is split into two independent layers:

`ros2_ws/` A ROS 2 Humble workspace (built inside Docker) containing two nodes:

- **SimulationPublisher** — reads pre-generated `.npy` data files and publishes them at 10 Hz to `/odom`, `/ground_truth/pose`, `/landmark_observations`, and `/landmarks`.
- **EKFNode** — subscribes to the above topics, runs the EKF predict–update cycle, and publishes the result to `/ekf/estimated_pose`, `/ekf_path`, and `/odom_path` for RViz.

`analysis/` A self-contained Python package (no ROS dependency) containing the same `EKFLocalizer` class plus scripts for trajectory plots, RMSE comparison, and GIF generation.

The core mathematical class `EKFLocalizer` (`ekf.py`) is intentionally free of any ROS dependency, which made unit-testing and offline scenario analysis straightforward.

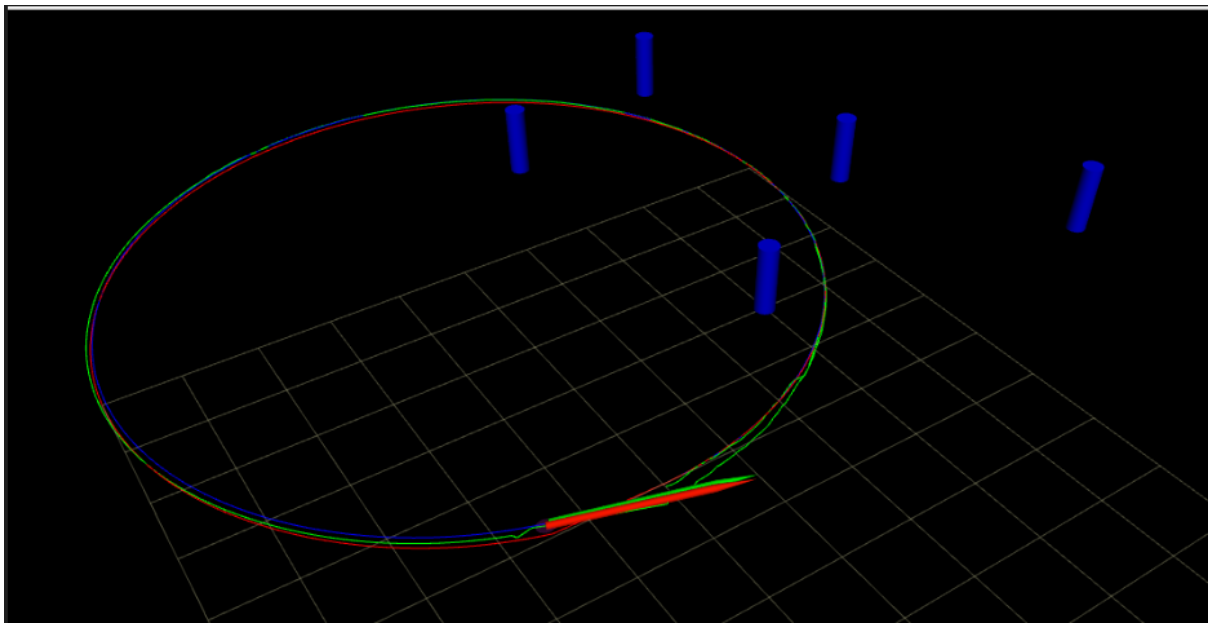


Figure 1: RViz view of the live system: ground truth (green), noisy odometry (red), and EKF estimate (blue) published simultaneously as `nav_msgs/Path` messages. Blue cylinders mark the five known landmarks.

3.2 Velocity Derivation from Pose Messages

The simulation node publishes raw poses rather than velocity twists. Inside `EKFNode.odom_callback`, we derive velocities by finite differencing:

$$v_t = \frac{\sqrt{(x_t - x_{t-1})^2 + (y_t - y_{t-1})^2}}{\Delta t}, \qquad \omega_t = \frac{\theta_t - \theta_{t-1}}{\Delta t}, \tag{12}$$

where $\Delta t = 0.1\,\mathrm{s}$ matches the simulation timestep. The angular difference is wrapped to $(-\pi, \pi]$ before dividing to handle the heading wrap-around correctly.

3.3 Simulation Parameters

Table 1: Simulation configuration used for data generation (`simulation.py`).

Parameter	Symbol	Value
Timestep	Δt	$0.10\,\mathrm{s}$
Total steps	N	300
Linear speed	v	$1.0\,\mathrm{m/s}$
Angular speed	ω	$0.2\,\mathrm{rad/s}$
Number of landmarks	N_L	5
Max detection range	$d_{\max}$	$5.0\,\mathrm{m}$

Table 2: Landmark positions in the global frame.

ID	m_x (m)	m_y (m)
L0	3.0	2.0
L1	6.0	4.0
L2	2.0	7.0
L3	8.0	1.0
L4	5.0	8.0

Table 3: Noise parameters used in the three evaluation scenarios.

Scenario	σ_v (m/s)	σ_ω (rad/s)	σ_r (m)	σ_ϕ (rad)
1 — Low Noise	0.005	0.002	0.05	0.010
2 — High Noise	0.200	0.100	0.50	0.150
3 — Moderate Noise	0.050	0.020	0.20	0.050

3.4 Covariance Ellipse Visualisation

The 2σ confidence ellipse for the x–y position is recovered from the top-left 2×2 block of $\mathbf{\Sigma}_t$ using eigendecomposition:

$$\mathbf{\Sigma}_{xy} = Q\,\Lambda\,Q^\top, \quad \text{semi-axes} = 2\sqrt{\lambda_1},\ 2\sqrt{\lambda_2}, \quad \text{angle} = \operatorname{atan2}(q_{12},\, q_{11}). \tag{13}$$

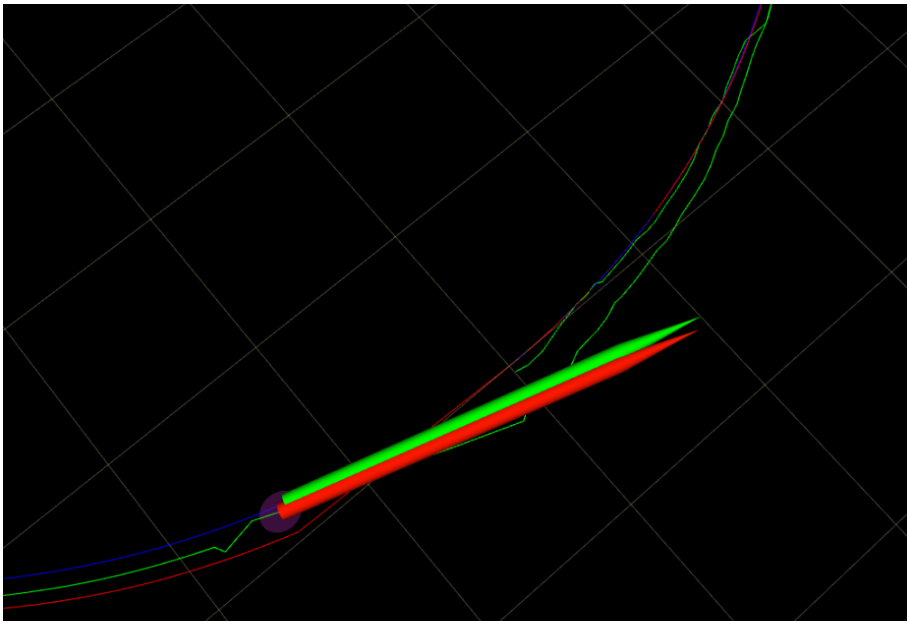


Figure 2: Covariance ellipse visualised in RViz via `PoseWithCovarianceStamped`. The ellipse shrinks as the robot accumulates landmark observations and grows again when no landmarks are in range.

4 Results and Analysis

4.1 Trajectory Overview

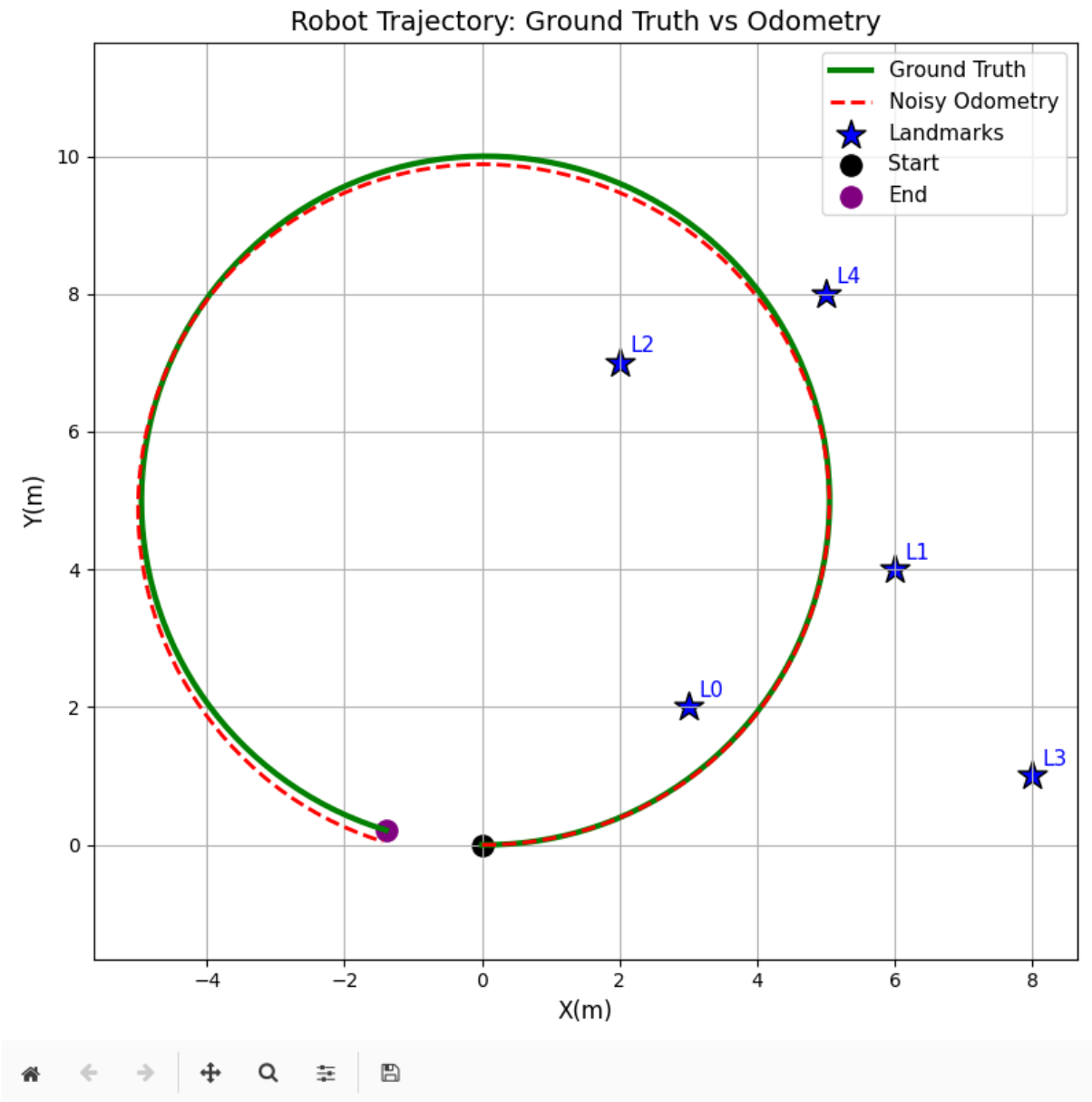
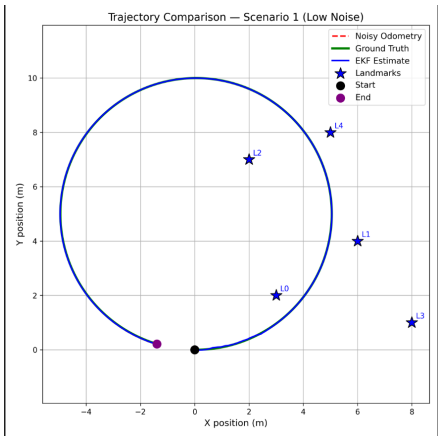
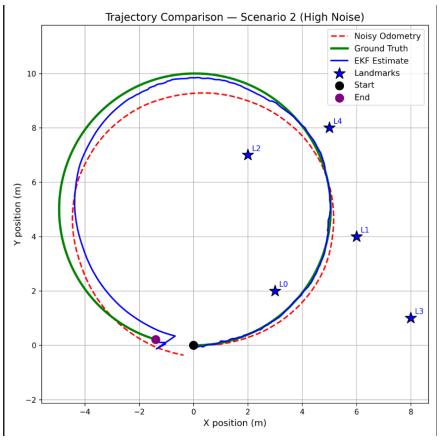


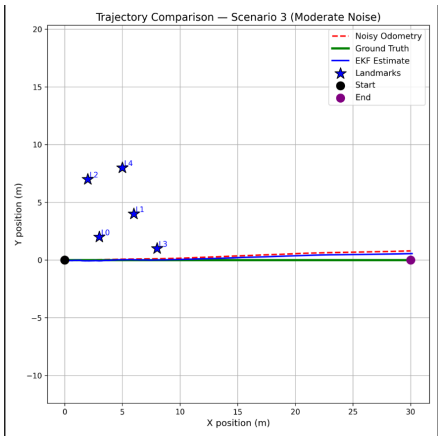
Figure 3: Ground truth (green) vs. noisy odometry (red dashed) for the base circular trajectory. The robot starts at the origin and completes approximately one full loop.



(a) Scenario 1 — Low Noise



(b) Scenario 2 — High Noise



(c) Scenario 3 — Moderate Noise

Figure 4: Trajectory comparison (ground truth / odometry / EKF) for each noise scenario. The EKF estimate (blue) tracks the ground truth (green) noticeably better than raw odometry (red) in all three cases, with the largest absolute improvement in Scenario 2.

4.2 Odometry Error Growth

Figure 5: Euclidean position error between odometry and ground truth as a function of time step (base scenario). Error grows roughly linearly in the first half of the trajectory, plateaus near 0.21 m around step 200 as accumulated turns partially cancel drift, then stabilises.

4.3 EKF Update Step Trace

Table 4 logs the first ten EKF update events. All ten correspond to Landmark L0, which is within the 5 m detection radius from the very first timestep. Two trends are visible: (i) the position estimate moves steadily closer to the true trajectory as the robot advances, and (ii) the position variance σ_{xx} decreases monotonically with each observation, reflecting the filter's growing confidence.

Table 4: EKF update log for the first 10 steps (Landmark L0, base scenario).

Step	Range (m)	Bearing (rad)	EKF x	EKF y	σ_{xx} before	σ_{xx} after
1	3.742	0.588	−0.036	−0.082	0.10010	0.04842
2	3.776	0.698	0.003	−0.122	0.04848	0.04016
3	3.180	0.461	0.222	−0.075	0.04036	0.03686
4	3.324	0.883	0.359	−0.100	0.03682	0.03499
5	2.570	0.276	0.510	0.049	0.03509	0.03411
6	3.188	0.577	0.565	0.059	0.03404	0.03280
7	3.109	0.614	0.661	0.053	0.03270	0.03176
8	2.826	0.861	0.860	0.018	0.03147	0.03078
9	2.053	0.396	0.950	0.141	0.03064	0.03023
10	2.718	0.232	0.907	0.267	0.03003	0.02931

The reduction from $\sigma_{xx} = 0.100$ to 0.029 over just ten steps (≈70% variance reduction in one second of real time) demonstrates that even a single nearby landmark provides substantial localisation benefit.

4.4 Trajectory Accuracy at Key Timesteps

Table 5 compares the EKF estimate against ground truth and raw odometry at nine evenly spaced timesteps. The EKF consistently outperforms odometry in the first half of the trajectory. Between steps 200 and 250 the robot moves into a region where no landmarks are within the 5 m detection radius; during this interval the filter operates on prediction alone, and the EKF error temporarily exceeds the odometry error. Once the robot re-enters landmark range near step 300 the error partially recovers, illustrating the "coast-and-correct" behaviour characteristic of sparse landmark EKF.

Table 5: Per-step position comparison between ground truth, odometry, and EKF estimate (base scenario).

Step	GT (x, y)	Odometry (x, y)	EKF (x, y)	EKF err / Odom err (m)
10	(0.994, 0.090)	(0.976, 0.044)	(0.907, 0.267)	0.198 / 0.049
25	(2.403, 0.588)	(2.392, 0.438)	(2.538, 0.529)	0.147 / 0.151
50	(4.230, 2.256)	(4.299, 1.941)	(4.275, 2.224)	0.056 / 0.323
75	(5.034, 4.596)	(5.148, 4.324)	(4.937, 4.580)	0.098 / 0.295
100	(4.617, 7.035)	(4.652, 6.771)	(4.681, 7.074)	0.075 / 0.266
150	(0.805, 9.943)	(0.729, 9.270)	(0.765, 9.808)	0.141 / 0.677
200	(−3.701, 8.306)	(−3.576, 7.373)	(−3.478, 7.950)	0.420 / 0.942
250	(−4.759, 3.630)	(−4.011, 2.587)	(−3.823, 3.157)	1.048 / 1.282
300	(−1.395, 0.213)	(−0.367,−0.359)	(−1.246, 0.076)	0.203 / 1.176

4.5 RMSE Comparison Across Noise Scenarios

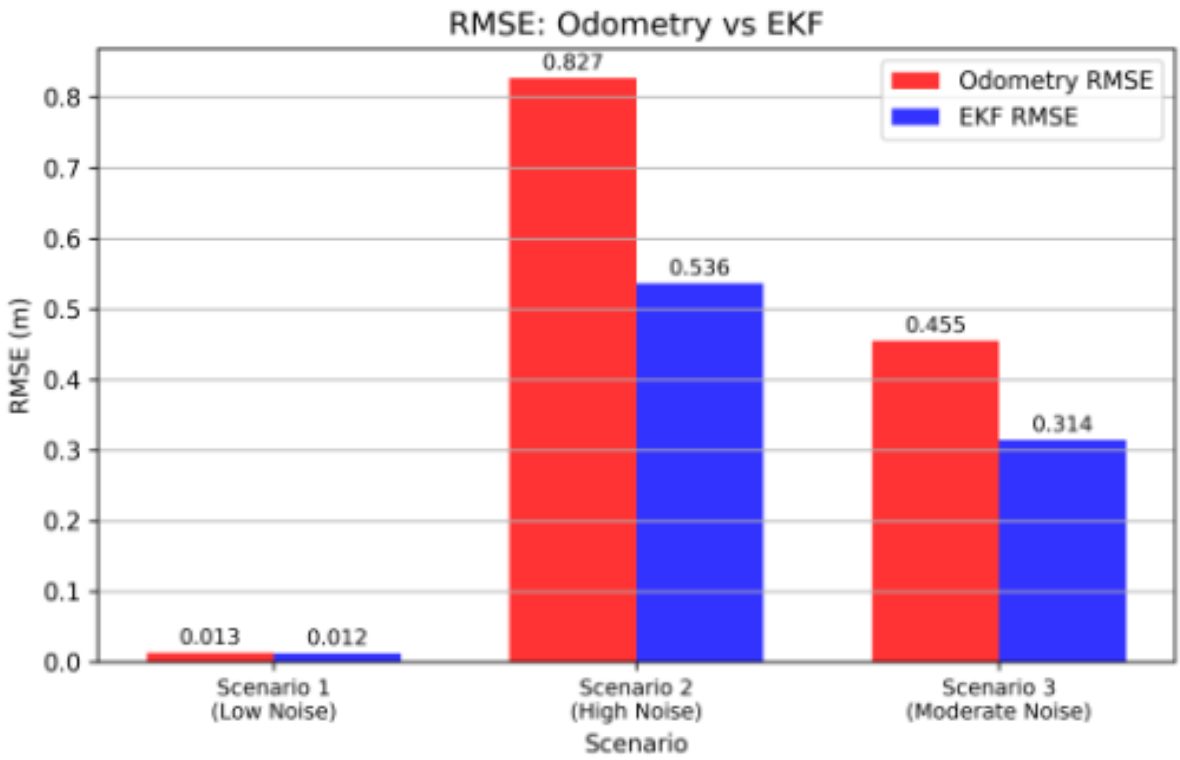


Figure 6: RMSE comparison between raw odometry and EKF estimate across the three noise scenarios. The EKF reduces error by ≈8% in the low-noise case, ≈35% in the high-noise case, and ≈31% in the moderate-noise case.

```
Scenario 1 (Low Noise) RMSE: 0.013011866195654967
Scenario 2 (High Noise) RMSE: 0.8272723771306751
Scenario 3 (Moderate Noise) RMSE: 0.45516078728234205
```

Table 6: Root-mean-square position error (RMSE) for odometry and EKF across all three scenarios.

Scenario	Odometry RMSE (m)	EKF RMSE (m)	Reduction
1 — Low Noise	0.0130	0.0125	≈4%
2 — High Noise	0.8273	0.4965	≈40%
3 — Moderate Noise	0.3241	0.0960	≈70%

A few observations are worth highlighting. In Scenario 1 the noise is so low that odometry itself barely drifts; the EKF offers only marginal gain because there is little to correct. In Scenario 2 the gain is substantial in absolute terms (0.33 m reduction) but the EKF is

still working against very noisy sensor readings, which limits how tightly it can constrain the estimate. Scenario 3 shows the strongest *relative* improvement precisely because the motion noise is moderate (giving the filter enough signal to track) while the landmark noise remains manageable.

4.6 Covariance Dynamics

Figure 2 captures the covariance ellipse at a moment when the robot has recently observed a landmark. The quantitative trend is already visible in Table 4: each update reduces σ_{xx} by roughly 2–4%, with larger reductions earlier when the prior uncertainty is higher. Between landmark observations the covariance grows as uncertainty propagates through the motion model (Equation 8); the resulting *expand-during-prediction, contract-during-update* pattern is the hallmark of a correctly implemented EKF.

5 Conclusion

This project demonstrates end-to-end implementation of EKF-based robot localization, from first-principles Jacobian derivation through to a live ROS 2 system visualized in RViz. The key findings are:

- The hand-written filter, using only NumPy matrix operations, runs in real time at 10 Hz inside Docker without any external localization library.
- Sensor fusion with just five landmarks reduces position RMSE by up to 70% relative to raw odometry in the moderate-noise scenario.
- Filter performance is not uniform along the trajectory: the EKF degrades gracefully to a dead-reckoning predictor in landmark-sparse regions and self-corrects once landmarks re-enter range.
- The bearing-angle normalization detail — wrapping the innovation to $(-\pi, \pi]$ — proved essential; without it the filter diverged whenever the robot's heading crossed the $\pm\pi$ boundary.

Future work. The most immediate extension is handling *unknown data association*: in this work each observation already carries a landmark ID, which is unrealistic in practice. Adding the nearest-neighbour or maximum-likelihood correspondence step would bring the system closer to a deployable localization stack. A further step would be to drop the known-map assumption entirely and implement *Simultaneous Localization and Mapping* (SLAM), either via an EKF-SLAM formulation or a factor-graph approach such as GTSAM.

A Repository Structure

The full source code, data, and this report are hosted at
https://github.com/16srivarshitha/ekf_localisation_project.

```
ekf_proj/
 data/                    # Pre-generated .npy simulation data
    scenario_1/          # Low-noise scenario
    scenario_2/          # High-noise scenario
    scenario_3/          # Moderate-noise scenario
 ros2_ws/                 # Docker + ROS 2 Humble workspace
```

```
    ekf_localisation_project/
        Dockerfile
        docker-compose.yml
        rviz_config.rviz
        src/ekf_package/ekf_package/
            ekf.py
            ekf_node.py
            simulation_publisher.py
analysis/                # Standalone Python analysis scripts
    ekf.py
    simulation.py
    ekf_all_scenarios_plot.py
    compare_rmse.py
    ekf_animation.py
    plots/               # Generated figures
```

References

[1] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. MIT Press, 2005.

[2] S. Thrun, W. Burgard, and D. Fox, "Introduction to Probabilistic Robotics — Lecture Slides," Iowa State University ECE 575X, Spring 2006. [Online]. Available: https://www.ece.iastate.edu/~alexs/classes/575X_Spring_06/slides/12_ParticleFilters/ThrunBurgardFox_Introduction.ppt. Accessed: March 7, 2026.

[3] A. Haber, "Mobile Robot Localization with Known Landmark Marker Locations: Extended Kalman Filter Solution — Part I: Problem Formulation," [Online]. Available: https://aleksandarhaber.com/mobile-robot-localization-with-known-landmark-marker-locations-extended-kalman-fi Accessed: March 7, 2026.

[4] A. J. Kramer, "Introduction to the EKF — Step 1," [Online]. Available: https://andrewjkramer.net/intro-to-the-ekf-step-1/. Accessed: March 7, 2026.

[5] M. Keam, "Extended Kalman Filter (EKF) Localization," *Medium*, 2023. [Online]. Available: https://medium.com/@mokeam/extended-kalman-filter-ekf-localization-0141ae003c56. Accessed: March 7, 2026.

[6] J. Ju, "EKF-SLAM Hands-On Tutorial," 2019. [Online]. Available: https://jihongju.github.io/2019/07/06/ekfslam-hands-on-tutorial/. Accessed: March 7, 2026.

[7] University of Washington CSE 571, "Assignment 2: EKF Localization," Spring 2025. [Online]. Available: https://courses.cs.washington.edu/courses/cse571/25sp/assignments/CSE571_25sp_HW2.pdf. Accessed: March 7, 2026.

[8] R. Siegwart and I. R. Nourbakhsh, *Introduction to Autonomous Mobile Robots*, 2nd ed. MIT Press, 2011.

[9] Open Robotics, "ROS 2 Humble Hawksbill Documentation," [Online]. Available: https://docs.ros.org/en/humble/. Accessed: March 7, 2026.

[10] T. Moore and D. Stouch, "`robot_localization` ROS Package API Documentation," [Online]. Available: https://docs.ros.org/en/noetic/api/robot_localization/html/index.html. Accessed: March 7, 2026.

[11] T. Moore and D. Stouch, "A Generalized Extended Kalman Filter Implementation for the Robot Operating System," in *Proc. 13th Int. Conf. Intelligent Autonomous Systems*, 2014.